API Design Developer Guide

Oct 28, 2026 · @kodebot

This guide explains how to design the HTTP endpoints of our microservices. It also explains how the Express front end calls them. Section 5.5 lists the Spring Boot feature to use for each problem.

All examples come from `manage-claim-service`. In this service, a caseworker creates a **claim** for a person. The caseworker adds **evidence** (documents such as payslips) and **notes** to the claim. At the end, the caseworker closes the claim. Each claim and each note is a separate record (a document in the database).

Terms used in this guide:

- **Express:** our front-end server. It is a Node.js app that builds the GOV.UK pages and calls the microservices.
- **Record:** one thing that a microservice stores, such as a claim or a note.
- **Version:** a number on every record. It goes up by one each time the record is saved. Every response that returns a record includes its `version`.
- **Design group:** the lead engineer, senior engineers and business experts. They approve design decisions.
- **Platform library:** our shared `service-platform` library. Every microservice uses it.
- **`@ClaimsRead` and `@ClaimsWrite`:** annotations from the platform library. They check that the caller has permission to use an endpoint.

1. Principles

1. **There are three kinds of endpoint that change data:** create a record, save details, and do a business action. Section 2.1 tells you which one to use.
2. **One save, one request.** When the user presses a button that saves, Express sends one request. The microservice saves everything in that request, or nothing. To show a page, Express can send many `GET` requests.
3. **Read endpoints are built around the data,** such as claims, notes and lookup lists. They are not built for one screen. Any screen can use them.
4. **Each kind of record has two response formats at most:** a short one for lists and a full one for one record.
5. **The browser never calls a microservice.** Only Express and other microservices call microservices.
6. **Each save goes to one microservice.** Express can read from several microservices to show one page. But it sends each save to one microservice only. If a process must change data in several microservices, the microservices do it with events (section 4).
7. **A microservice calls another microservice only to read or check something** (section 4). A business action can also send its single change to the system that stores the data (section 2.5).
8. **Every microservice checks permissions itself,** even when the call comes from our own Express app.
9. **Every call has a time limit.** If a microservice is slow or down, the rest of the page still works (section 6).
10. **The build checks every API change.** The build fails if a change would break Express (section 5.4).

2. Endpoints that change data

2.1 Which endpoint for which change

Our front end follows the GOV.UK Design System. The user saves in one of two ways:

- **After a Check answers page:** the user answers questions on several pages. Then the user checks the answers on a **Check answers** page, where each answer has a "Change" link. Then the user presses one button, such as "Accept and send".
- **On a single page:** the user changes something on one page and presses "Save". There is no Check answers page.

Both ways send one request. The pages do not decide the endpoint. **What the user saves** decides it. Use this test, in order, and stop at the first "yes":

1. **Does the save create a new record?** Use **create**: `POST` to the collection. Example: `POST /claims`.
2. **Is the change a business action?** It is a business action if any of these is true:
   - The business process names the change as its own step. Examples: close a claim, verify evidence, change address, change name, report a change of circumstances.
   - The change needs more than the new values, such as the date it takes effect, a reason or evidence.
   - The change sets a field that only a business rule may change, such as `status` or `closedAt`.

   Use a **business action**: `POST` to a URL that ends with the action name. Examples: `POST /claims/{claimId}/close`, `POST /customers/{customerId}/change-address`.
3. **Otherwise, it is a details change.** The user changes one or more details in a group. A **group** is a set of details that the screen shows together, such as the claim's contact preferences (contact method, phone number and email address). Use **save details**: `PUT` to the group's URL. Example: `PUT /claims/{claimId}/contact-preferences`. Send every field in the group, changed or not, and the record's `version`.

If you are not sure about step 2, ask the business.

Example of save details: the caseworker changes the contact method to email and adds an email address. Then the caseworker presses "Save". Express sends the whole group, changed or not:

```
PUT /claims/9f1c.../contact-preferences
{ "version": 7, "contactMethod": "EMAIL", "phoneNumber": "", "emailAddress": "jane@example.com" }

204 No Content
```

The microservice then does these things:

1. It checks the `version`. If someone else saved the claim after the screen loaded, it refuses the request with `409`.
2. It compares each new value with the stored value. The `version` check makes sure that the stored values are the ones the user saw. So every difference is the user's change. Express never sends a list of changed fields.
3. It checks the business rules. For example, you cannot change a closed claim.
4. It saves all the changes together.
5. It writes the names of the changed fields to the audit log. It does not write the values, because they are personal data.
6. It publishes one event, `ContactPreferencesChanged`, that lists the changed fields.

Rules for groups of details:

- Each field belongs to one group only.
- A group never has a field that only a business action may change, such as `status` or `closedAt`.
- If the user changes only one field, Express still sends the whole group. Express already has every value, because it loaded the group to show the page.
- Do not use `PATCH`, because Express always sends the whole group.
- Do not make `PUT /claims/{claimId}` that replaces the whole claim.
- Make one endpoint for each group. Do not make one for each question page or each field, such as `update-phone`.
- A details change can still have effects inside the microservice, such as a new calculation. The microservice handles them. Express does not need to know about them.

Does this create too many endpoints? No. The business decides the groups and the business actions, not the number of pages. A complex record such as a claim has about 10 to 20 of these endpoints.

2.2 One save, one request

A new claim is collected over several question pages. These pages can include a first note. Express keeps the answers until the user presses "Accept and send" on the Check answers page. Then Express sends **one** request with the claim and the note:

```
POST /claims
{ "claimType": "STANDARD", "fullName": "Jane Doe", "dateOfBirth": "1990-01-01",
  "notes": [ { "text": "Call back Monday", "category": "CONTACT", "actionRequired": true } ] }

201 Created
Location: /claims/9f1c...
{ "claimId": "9f1c...", "noteIds": ["4ab2..."] }
```

The microservice saves the claim and the note in one database transaction. Both are saved, or neither is saved.

Do not send `POST /claims` and then `POST /claims/{claimId}/notes` for one save. The second request can fail. Then the claim exists without the note that the user typed. Express has no safe way to undo the first request.

Sometimes the user does two things at different times. For example, the user creates a claim today and adds a note tomorrow. Then use the two separate endpoints.

`POST /claims` accepts 10 notes at most. If a request has more, the microservice refuses it with `400`. This keeps the transaction small.

2.3 Retrying safely

A request can time out after the microservice has saved the data. If Express sends the request again, it could create a second claim. These rules stop this:

- **Do not create a record twice.** `POST /claims` and `POST /claims/{claimId}/notes` accept an `Idempotency-Key` header. Express makes a random ID when it builds the page that has the submit button. It puts the ID in a hidden field, and sends it every time the user submits that page. If the microservice has already processed that ID, it returns the first result. It does not create a second record.
- **Do not overwrite another user's change.** Every request that changes an existing record sends the record's `version` in the body. Express got this `version` when it loaded the record. If the stored version is different, someone changed the record after the screen loaded. The microservice refuses the request with `409`. A repeated request also gets `409`, so the change is not made twice.
- **Adding an item inside a record is a change to that record.** For example, evidence is stored inside the claim. So `POST /claims/{claimId}/evidence` sends the claim's `version`. It does not use an `Idempotency-Key`.
- **Do not retry changes automatically.** If a change fails or times out, Express shows the error. The user checks the record and then submits again.

2.4 What these endpoints return

Result	Status	Body
A record was created	`201 Created`, with a `Location` header that has the new record's URL	The new IDs
Details or an action were saved	`204 No Content`	Empty. If Express needs the record, it reads it with `GET`
The record changed after the screen loaded	`409`	Error code `CONCURRENT_UPDATE`
A business rule refused the change	`422`	The error `code` names the rule

Section 5.3 lists every status code and what Express does.

2.5 Business actions that call another system

Some business actions send their change to another system. For example, the central customer service stores the customer's name, date of birth and address. Our business process has one screen for each of these changes, with its own Save button. Each change is one business action, and it makes one call to the other system. So a save can never half-succeed.

```
GET /customers/{customerId}/address
200 OK
{ "customerId": "...", "version": "\"v7\"", "line1": "2 Low Road", "town": "York", "postcode": "YO1 7HH" }

POST /customers/{customerId}/change-address
{ "version": "\"v7\"", "line1": "1 High Street", "town": "Leeds", "postcode": "LS1 1AA" }
204 No Content
```

Here the `version` is the other system's version of the record (its ETag). It is text, not a number. Express keeps it and sends it back with the change.

Rules:

1. **Make one business action for each change,** such as `change-name` or `change-address`. Do not make one endpoint that calls the other system several times. If the second call fails, the first change is already made, and we cannot undo it.
2. **Check our own rules first.** Validate the request and check permissions. Then call the other system.
3. **Call the other system once, and change nothing else in the same request.** If we keep a copy of the data, the copy is only a cache. Update it after the other system confirms the change, or when its event arrives.
4. **Send the version on.** Our microservice passes the `version` to the other system, for example in an `If-Match` header. If the other system has no version, the last save wins. Write this risk in the architecture decision record (ADR).
5. **Turn the other system's answers into ours:**
   - It refuses the change: return `422` with our own code, such as `ADDRESS_REJECTED`.
   - Its version has changed (it answers `412`): return `409` with the code `CONCURRENT_UPDATE`.
   - We cannot reach it, or it answers `503`: return `503`. Nothing was changed.
   - It does not answer in time: return `504` with the code `OUTCOME_UNKNOWN`. The change may or may not be made. Express asks the user to check the details before trying again.
6. **Never retry the change automatically.** If the first attempt worked, a repeat fails with `412`. Then the user is told about a conflict that did not happen. Reads can be retried.
7. **Write our audit record after the other system confirms.** Record who asked for the change and what changed, for example "address".

3. Endpoints that read data

Reading data does not change anything. So read endpoints are built around the records. Any screen can call the endpoints it needs and combine the results. Every read endpoint is one of these five kinds:

Kind	Example	What it must do
One record	`GET /claims/{claimId}`	Return the full format. If the record does not exist, return `404`.
A list, with filters	`GET /claims?status=OPEN&page=0&size=20`	Always sort the results. Always return one page at a time. `size` is 100 at most. Each filter field must have a database index, so the query stays fast when the data grows.

Kind	Example	What it must do
A record's children	`GET /claims/{claimId}/notes?page=0&size=20`	If the claim does not exist, return `404`. Do not return an empty list.
A lookup list	`GET /reference-data/CLAIM_TYPE`	Return an object with an `items` list. Each item has a `code`, a `label` and an `active` flag. Send `Cache-Control: max-age=300`, so Express can keep the list for 5 minutes.
A calculated answer	`GET /claims/{claimId}/entitlement?from=2026-01-01&to=2026-01-31`	Return a result that the microservice works out with business rules. Here, it is the amount owed for those dates. This is the only kind of read endpoint that is built for one question.

Response formats. Each kind of record has two formats:

- **Summary.** Use it in lists. It has the ID and the few fields that a list shows. For a claim: ID, type, status and created date.
- **Detail.** Use it for one record. It has every field that the caller is allowed to see.

Sometimes a list screen needs one more field. Then add the field to the summary format. Do not make a third format for that screen. If you do, the microservice must change every time a screen changes.

Do not make one call for each row. A list of 20 claims must not cause 20 more calls for the details of each row. Do one of these things instead:

- Add the field to the summary format.
- Get all the rows in one call: `GET /claims?ids=9f1c...,7a2d...` (100 IDs at most).

Lookup lists. Express shows the `label` of each value and stores its `code`. Do not write labels in Express code. Labels can change, but codes never change. A value with `active: false` still shows on old records. But forms do not offer it.

Permissions. Read endpoints also need a permission check (`@ClaimsRead`), because they return personal data.

4. Processes that change data in several microservices

A database transaction cannot cover two microservices, because each microservice has its own database. So a process that changes data in several microservices is a series of steps. Each step changes data in one microservice. Then it tells the next microservice with an event.

Example: when a claim is closed, the payment microservice must stop future payments.

1. Express calls `POST /claims/{claimId}/close` on `manage-claim-service`. This is the only call that Express makes.
2. `manage-claim-service` closes the claim. In the same database transaction, it stores a `ClaimClosedV1` event. The platform library then sends the event to the other microservices. The claim now shows `paymentsStopped: false`.
3. The payment microservice receives `ClaimClosedV1`. It stops the payments in its own database transaction.
4. When the payments have stopped, the payment microservice sends `PaymentsStoppedV1`. Then `manage-claim-service` sets `paymentsStopped: true`. If the payment microservice cannot stop the payments, it sends `PaymentStopFailedV1`. Then `manage-claim-service` creates a task for a caseworker to fix the problem.

The task for a caseworker in step 4 is a **compensation**. We cannot undo the claim closure in step 2. So a new business action handles the failure. The business decides what this action is.

Rules

1. **One microservice owns the process.** We call it the **owner**. Express calls only the owner, once. Express never calls a second microservice to finish the job.
2. **Before the first change, a microservice can call another microservice to read or check something.** Example: before it creates a claim, `manage-claim-service` asks the eligibility microservice if the person is eligible. Inside a process, a microservice never calls another microservice to change data. It sends an event instead.
3. **Each step saves its change and its event together.** So an event is never lost. And an event is never sent for a change that was not saved.
4. **The process design says what happens when a step fails.** For example, cancel a payment, or create a task for a caseworker. The design group agrees this design.
5. **Every step must be safe to run twice,** because an event can arrive more than once. The platform library records the ID of every event that it processes, in the same transaction as the change. It skips an event if it has already recorded its ID.
6. **The owner records the progress of the process.** Express shows it to the user, for example "Waiting for payments to stop". An alert is raised if a process is stuck for too long.
7. **Every process like this needs approval from the design group,** and an architecture decision record (ADR).

Two ways to link the steps

	Each microservice reacts to the last event	**The owner directs every step**
Also called	Choreography	Orchestration
How it works	Each microservice listens for an event. It does its step. Then it sends its own event.	The owner keeps a record of the process. It sends each step to the next microservice as a command message, not as an HTTP call. It waits for the reply event. Then it decides the next step.
Use it when	There are 2 or 3 steps, and failures are simple	There are many steps or deadlines. Or the business needs to see the progress of each step.
Example	Close a claim, then stop payments	Award a claim: create the award, then set up payments, then send a letter

Start with the first way. Change to the second way when the chain of events becomes hard to follow.

What the owner's endpoint returns

- **The user needs only the first step,** such as closing the claim: return `204` as usual. Show the progress of later steps as a field on the record, for example `paymentsStopped: false`.
- **The user must wait for the whole process:** return `202 Accepted` with a `Location` header. The header points to a status URL, for example `/claims/{claimId}/award`. A `GET` on this URL returns `IN_PROGRESS`, `COMPLETED`, or `FAILED` with a reason. Express shows the status, and checks again when the user refreshes the page.

5. Conventions

5.1 HTTP methods

Method	**Use it for**	**Example**
`GET`	Reading data. A `GET` never changes anything	`GET /claims/{claimId}`
`POST` to a collection	Creating a record (2.1, step 1)	`POST /claims`

Method	Use it for	Example
`POST` to an action URL	A business action (2.1, step 2)	`POST /claims/{claimId}/close`
`POST` to a search URL	A search by personal details. It only reads data (5.2)	`POST /claims/search`
`PUT`	Saving details (2.1, step 3). Send every field in the group, and the `version`	`PUT /claims/{claimId}/contact-preferences`
`PATCH`	Not used. Express always sends the whole group with `PUT`	—
`DELETE`	Not used for business records. To end a record, use an action such as `close` or `withdraw`, so its history is kept. Use `DELETE` only if the design group agrees that a record can be removed for ever	—

5.2 URLs and JSON

- Collections are plural nouns: `/claims`, `/claims/{claimId}/notes`.
- An action is a verb, or a verb and a noun, at the end of the URL. Use lower case and hyphens: `/close`, `/resolve-action`, `/reopen`.
- Nest one level at most below the main record. `/claims/{claimId}/notes/{noteId}` is fine. `/claims/{claimId}/notes/{noteId}/attachments/{attachmentId}` is too deep. Use `/attachments/{attachmentId}` instead.
- Filters, sorting and paging are query parameters: `?status=OPEN&sort=createdAt,desc&page=0&size=20`.
- IDs are UUID strings. Express sends them back unchanged. Express never builds IDs and never reads meaning from them.
- A response is always a JSON object, never a JSON array. Put a list inside an object, such as `{ "items": [...] }`.
- We use plain JSON, not JSON:API. The GDS standards suggest JSON:API only when an organisation has no standard of its own. This guide is our standard.
- JSON field names are camelCase, such as `claimId`. The GDS standards ask for consistent names. camelCase is our choice.
- Dates look like `2026-01-31`. Date-times are in UTC and end with `Z`: `2026-01-31T09:00:00Z`. Money is a number with two decimal places: `242.86`.
- Never put personal data in a URL, such as names, dates of birth or National Insurance numbers. URLs are written to logs. To search by these details, send them in the body of `POST /claims/search`.

5.3 Status codes and errors

Every microservice returns the same status codes in the same situations. The platform library does this for you.

Status	Meaning	What Express does
`400`	The request is not valid. The `errors` list in the body names each wrong field	Shows each error next to its field
`401`	The user is not signed in, or the session has ended	Sends the user to sign in again through the gateway
`403`	The user is signed in but is not allowed to do this	Shows "You do not have permission"
`404`	The record does not exist	Shows a "not found" page
`409`, code `CONCURRENT_UPDATE`	Someone changed the record after the screen loaded	Shows the 409 page (6.1). Never retries automatically
`409`, code `REQUEST_IN_PROGRESS`	The first request with this `Idempotency-Key` has not finished	Asks the user to wait, and then to check the record

Status	Meaning	What Express does
`422`	A business rule refused the change. The `code` names the rule, such as `CLAIM_HAS_OPEN_ACTIONS` or `ADDRESS_REJECTED`	Shows the message for that `code`
`422`, code `IDEMPOTENCY_KEY_REUSED`	The same `Idempotency-Key` came with a different request. This is a bug in Express	Shows an error page
`500`	An unexpected error	Shows an error page with the `traceId`
`503`	A system that the microservice needs is not available. Nothing was changed	Shows "Try again later". A read (`GET`, or `POST /claims/search`) can be retried once
`504`, code `OUTCOME_UNKNOWN`	Another system did not answer in time. The change may or may not be made (2.5)	Shows "We could not confirm your change. Check the details before you try again." Never retries automatically

All error bodies have the same format. This is the "problem details" standard (RFC 9457):

```
{ "status": 422, "title": "Business rule violated", "code": "CLAIM_HAS_OPEN_ACTIONS",
  "detail": "Claim 9f1c... has 2 notes that need action", "traceId": "4bf92f35..." }
```

- Express chooses the message for the user from the `code`. The `detail` text is for developers. Never show it to users.
- `traceId` identifies the request in the logs of every microservice that it touched. Error pages show it, so support can find the request.
- Error bodies never contain personal data.

5.4 Changing an API without breaking Express

Microservices and Express are released at different times. So a microservice must never release a change that the current Express cannot handle.

- **Safe at any time:** adding an endpoint, adding an optional field to a request, adding a field to a response. Express must ignore response fields that it does not know.
- **Breaking:** removing or renaming a field, changing a field's type or meaning, making an optional field required, changing a URL. Make these changes in three releases:
    1. The microservice adds the new field or endpoint. It keeps the old one.
    2. Express changes to the new one.
    3. The microservice removes the old one.
- Every path starts with the major version, such as `/v1`. This follows the GDS API technical and data standards. The examples in this guide leave it out. Use the three-release change when you can. If you cannot, add `/v2` endpoints next to the `/v1` endpoints. Then add two headers to `/v1` responses: `Deprecation`, and `Sunset` with the date when `/v1` will be removed.
- Each microservice describes its API in `contracts/openapi.yaml`. Update this file in the same pull request as the code. The build compares the file with the released version. The build fails if a change is breaking. Express generates its TypeScript client from these files. So if a field is removed, Express does not compile.

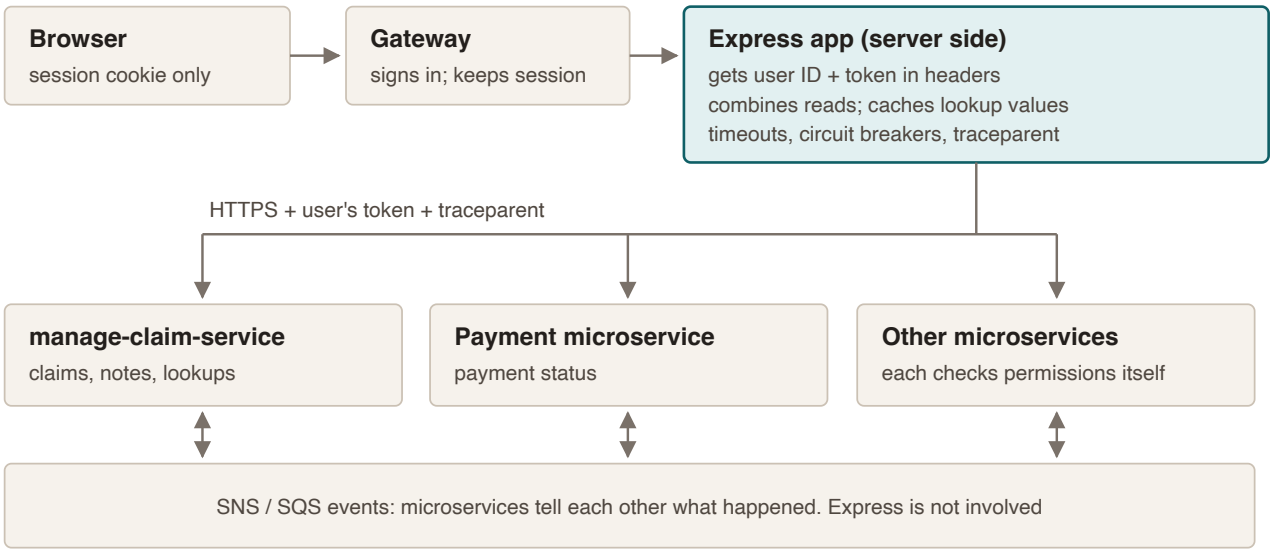
5.5 Which Spring Boot feature to use

Use these features for these problems. Do not build your own version.

Problem	Use	Notes
Endpoints and JSON bodies	`@RestController` with `@GetMapping`, `@PostMapping` and `@PutMapping`. One Java `record` for each request and each response	Never use the stored entity as a request or response
Checking a request (`400`)	Bean Validation: `@Valid` on the body, and `@NotNull`, `@NotBlank` or `@Size` on the record's fields	The platform library turns failures into the `errors` list
Permissions (`403`)	Spring Security resource server (JWT), with `@PreAuthorize` through `@ClaimsRead` and `@ClaimsWrite`	Put one on every endpoint. The build checks this
Saving several changes together (2.2)	`@Transactional` on the application service method. Use `TransactionTemplate` when part of the method must run outside the transaction, such as a call to another service	Keep transactions short
Refusing a change from an old screen (`409`)	Spring Data `@Version` on the stored record, plus the platform's `requireVersion` check	Both become `409 CONCURRENT_UPDATE`
`Idempotency-Key` (2.3)	The platform library's `IdempotentRequests`	Spring has nothing built in for this
`201` and `204` responses	`ResponseEntity.created(location)` and `ResponseEntity.noContent()`	
Error bodies (5.3)	`ProblemDetail` (RFC 9457), built into Spring, through the platform's `GlobalExceptionHandler`	Throw a domain exception. Never write an error handler in a feature
Lists with paging (3)	Spring Data `Pageable` and `Page`, with a query method such as `findByStatus(status, pageable)`	Add a database index for every filter
Caching lookup lists (3)	`ResponseEntity.ok().cacheControl(CacheControl.maxAge(Duration.ofMinutes(5)))`. On the server: `@Cacheable` on the read	
Calling another service (2.5, 4)	An `@HttpExchange` interface (`@GetExchange`, `@PostExchange`, `@PutExchange`, `@RequestHeader("If-Match")`), created by the platform's `HttpClientFactory`	Gives time limits and tracing
Retries and circuit breakers	Resilience4j, through the platform's `ResilientCalls`. Use `callWithRetry` for reads and checks. Use `callOnce` for changes	`callOnce` never retries. It returns `503` or `504` (2.5)
Sending events (4)	Spring Data `@DomainEvents`: events go out when the record is saved. Spring Modulith's event publication registry keeps each event in the database until it is sent (this is called the outbox)	Never send a message directly from a service method
Receiving events (4)	Spring Cloud AWS `@SqsListener`, through the platform's `InboxMessageHandler`	It skips events that were already processed

6. The Express front end

Every request from the browser goes to our gateway first. The gateway signs the user in and keeps the user's session. Then it sends the request to Express, with the user's ID and access token in request headers. Express builds the page on the server and calls the microservices that it needs. So Express is already our backend-for-frontend (BFF). A BFF is a server-side layer built for one front end. We do not add another layer.



Browser → Express → microservices: the browser talks only to Express, and Express calls each microservice. The gateway signs the user in and passes the user's ID and token to Express. Express calls each microservice over HTTPS with the user's token and traceparent, and each microservice checks permissions itself. The microservices tell each other what happened with SNS and SQS events, and Express is not involved.

Rule	What to do
The browser never calls microservices	The browser holds only the gateway's session cookie. Only Express knows the microservice addresses
Express trusts only the gateway	Express accepts requests only from the gateway, and the network enforces this. The gateway removes any identity headers that the browser sends. Express does not sign users in or keep sessions
Express passes on the access token	Express gets the user's access token from the gateway in a header. It sends the token with every call to a microservice. It never sends the token to the browser
Microservices check permissions	Microservices get the user's identity from the access token, never from a user-ID header. You can hide a button that the user cannot use. This helps the user, but it is not security. The microservice must still refuse the request with `403`
Read from several, change one	A page can call several microservices to read data. Make the calls in parallel with `Promise.allSettled`, so one failed call does not stop the others. Send each save as one request to one microservice (section 4)
Every call has a time limit	All calls go through one shared HTTP client module. The time limit is 2 seconds for reads and 5 seconds for changes. If a read (`GET`, or `POST /claims/search`) times out or gets `503`, retry it once. Never retry a change automatically
Stop calling a failing microservice	The shared client uses a circuit breaker, for example the `opossum` package. After several failures, it stops calling that microservice for a short time. It fails at once, so pages do not hang
Show what you can	For each page, decide which data is essential. If other data fails, show the rest of the page with a message. Example: if the payment status fails on the claim page, show "Payment details are not available right now". Show an error page only if essential data fails
Pass on the trace ID	OpenTelemetry adds the `traceparent` header to every call. So you can follow one user action through the logs of every microservice
Cache lookup lists only	Keep a lookup list for as long as its `Cache-Control` header says. Never cache claims, notes or other business records. Users must see the latest version
Handle errors in one place	One error handler turns the status and the `code` into what the user sees (section 5.3)

6.1 Forms and GOV.UK patterns

- **Pages work without JavaScript.** This is GOV.UK progressive enhancement. Every form is a normal HTML form that posts to Express. Express calls the microservices from the server. JavaScript can improve a page, but nothing can depend on it.
- **Express keeps the answers until the user saves.** While the user moves through question pages and "Change" links, Express stores the answers in its own server-side session store. It never stores them in the browser or in a microservice. Only a button that saves sends a request to a microservice. If users must be able to stop and come back later, ask the design group first.
- **After a successful save, redirect.** Express answers the form post with a redirect to the confirmation page. This is called post/redirect/get. A page refresh then cannot submit the form twice. The confirmation page shows the reference that the microservice returned, such as the new claim's ID.
- **Show validation errors with the GOV.UK error pattern.** Express shows an error summary at the top of the page and a message next to each field. The microservice's `400` response lists each wrong field in `errors`, with the same field names as the request. So Express can put each message in the right place. Express writes the message text in GOV.UK style, by field and error type.
- **A `409` has its own page.** The page says that someone else changed the record. It shows the latest version. It asks the user to make their changes again.

Later, we may add very different front ends, such as a public website or a mobile app. Then the design group will decide if we need a separate BFF layer.

7. Common mistakes

Mistake	Correct approach
`PATCH`, or `PUT /claims/{claimId}` that replaces the whole claim and its status	Use the test in 2.1 to choose create, business action or save details.
An endpoint for each question page or field, such as `POST /claims/{claimId}/update-phone`	One `PUT` for the whole group of details, such as `PUT /claims/{claimId}/contact-preferences` (2.1)
An endpoint named after a screen	Name it after the action or the data (2.1, 3)
Two requests for one save	One request with everything (2.2)
A create request without an `Idempotency-Key`, or a change without a `version`	Send them as described in 2.3
Express changes data in two microservices for one action	Call the owner microservice once. The others react to its events (4).
A new response format for one screen	Add the field to the summary or detail format (3)
One extra call for each row of a list	Add the field to the summary format, or get the rows by a list of IDs (3)
A list without paging, sorting or a database index	Use paging (100 at most), sort, and index every filter (3)
An endpoint without a permission check	Add `@ClaimsRead` or `@ClaimsWrite`, and a test that expects `403`
The browser calls a microservice	Move the call into Express (6)
A call without a time limit, or a change retried automatically	Use the shared HTTP client. Never retry changes automatically (6).
Removing or renaming a field in one release	Use the three-release change (5.4)
Personal data in a URL	Send it in the body of `POST /claims/search` (5.2)
One endpoint that sends several changes to another system, such as name and address together	One business action for each change, with one call each (2.5)

Sources: GOV.UK Design System: Check answers, GDS API technical and data standards, GOV.UK Service Manual: progressive enhancement.